Reading App.tsx (1/3)

The Structure: Data, Logic, and the Parent Component

Roadmap

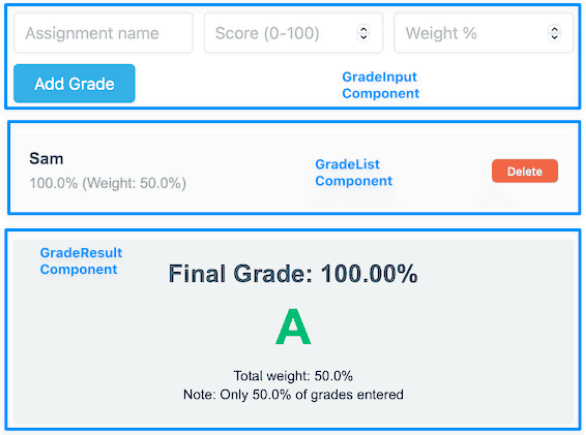
We read `App.tsx` in three parts:

1. **4-3 (this part)**: types, pure functions, and the parent component `GradeCalculatorApp`
2. **4-4**: the child components `GradeInput`, `GradeList`, `GradeItem`, `GradeResult`
3. **4-5**: persistence with `localStorage` (extra), break the code on purpose, and judge it

The Component Tree

From now on, we discuss all the React components of GradeCalculatorApp:

- GradeCalculatorApp
    i. GradeInput
    ii. GradeList
        a. GradeItem
    iii. GradeResult



IMPORTANT

The first step in high-quality software design is to define the **data model**.

- We use TypeScript `interface` to define the two data models that are created, processed, and displayed in this application.

Data Model: Types

```
interface Grade {
  readonly id: number;
  name: string;
  score: number;
  weight: number;
}
interface GradeSummary {
  percentage: number;
  totalWeight: number;
}

type LetterGrade = 'A' | 'B' | 'C' | 'D' | 'F';

const MAX_WEIGHT = 100;
const WEIGHT_EPSILON = 1e-9;
const STORAGE_KEY = 'grades';
```

Same ideas as the TypeScript example.

`WEIGHT_EPSILON` allows a tiny error because `0.1 + 0.2 !== 0.3`.

IMPORTANT

It is also important to separate the **business logic (algorithm)** from the **GUI**.

- Here, the calculations are pure functions: they take inputs and return values, without using React or the DOM.
- This makes them easy to test.

Pure Functions: Logic Without React

The calculation is **not** inside a component. These functions use no React and no DOM.

```
export function getLetterGrade(percentage: number): LetterGrade {
  if (percentage >= 90) return 'A';
  if (percentage >= 80) return 'B';
  if (percentage >= 70) return 'C';
  if (percentage >= 60) return 'D';
  return 'F';
}
```

Returns null when there is nothing to calculate

```
export function calculateFinalGrade(
  grades: Grade[]
): GradeSummary | null {
  const totalWeight = grades.reduce((sum, g) => sum + g.weight, 0);
  if (totalWeight === 0) return null;

  const weightedSum =
    grades.reduce((sum, g) => sum + g.score * g.weight, 0);
  return { percentage: weightedSum / totalWeight, totalWeight };
}
```

Test Without a Browser or React

The same example as before: 100 (50%) and 80 (50%).

```
calculateFinalGrade([
  { id: 1, name: 'HW', score: 100, weight: 50 },
  { id: 2, name: 'Exam', score: 80, weight: 50 },
]);
// { percentage: 90, totalWeight: 100 }  -> getLetterGrade(90) is 'A'

calculateFinalGrade([]);   // null
```

Because `calculateFinalGrade` is exported, it is also available in the browser console as `AppModule.calculateFinalGrade(...)`.

The Main Component: GradeCalculatorApp

Step 1: The State

The `grades` state lives here, in the parent, and is shared by all children.

```
function GradeCalculatorApp() {
  const [grades, setGrades] = useState<Grade[]>([]);
```

- We start with an empty list.
- The real `App.tsx` also loads and saves the grades (see 4-5, "Adding Persistence"). First, we focus on the components.

Step 2: Add a Grade

Adding a new grade. It returns an error message, or `null` on success:

```
const addGrade = (gradeData: Omit<Grade, 'id'>): string | null => {
  const currentTotalWeight =
    grades.reduce((sum, g) => sum + g.weight, 0);
  if (currentTotalWeight + gradeData.weight
      > MAX_WEIGHT + WEIGHT_EPSILON) {
    return `Total weight would exceed 100% ` +
           `(current: ${currentTotalWeight}%).`;
  }

  setGrades((previous) => {
    const nextId = Math.max(0, ...previous.map((g) => g.id)) + 1;
    return [...previous, { ...gradeData, id: nextId }];
  });
  return null;
};
```

- The new id is the largest id + 1, so ids stay unique even after a reload.
- We never change the old array (`push`). We create a **new** array with `[...previous, newGrade]`, so React notices the change (see the array rule in the React theory part).
- `previous` is the latest state, so we use the function form `setGrades((previous) => ...)`.

Delete the grade with the id:

```
const deleteGrade = (id: number) => {
  setGrades((previous) => previous.filter((g) => g.id !== id));
};
```

Step 3: Return JSX

```
  return (
    <div className="app">
      <h1>Grade Calculator (React Edition)</h1>
      <GradeInput onAdd={addGrade} />
      <GradeList grades={grades} onDelete={deleteGrade} />
      <GradeResult grades={grades} />
    </div>
  );
}
```

- The state lives in the closest common parent of the components that need it. This is called **lifting state up**.
- Data flows **down** as props (`grades`), and events flow **up** through callback props (`onAdd`,

JSX Becomes HTML

All the HTML tags are generated from React JSX. For example, `<h1>Grade Calculator (React Edition)</h1>` in JSX becomes the `<h1>` tag in the HTML page.

- HTML requires `class="grade-item"`, but JSX requires `className="grade-item"` (`class` is a reserved word in JavaScript).
- `{ }` embeds a JavaScript expression: `<strong>{grade.name}</strong>`.
- `condition && <tag/>` shows the tag only when the condition is true: `{error && <div className="error">{error}</div>}`.

The GradeCalculatorApp has three React Components

- GradeInput gets three inputs and has the Add button (the `addGrade` function is called)
- GradeList displays the grades (the `deleteGrade` function is called from each Delete button)
- GradeResult displays the final and letter grade (it only reads `grades`)

```
<GradeInput onAdd={addGrade} />
<GradeList grades={grades} onDelete={deleteGrade} />
<GradeResult grades={grades} />
```

Key Points of This Part

- **Types** describe the data (`Grade`), and **pure functions** do the calculation. Neither uses React.
- The state `grades` lives in the parent, `GradeCalculatorApp`.
- Data flows **down** as props. Events flow **up** as callbacks.
- JSX describes the UI: `className`, `{ }`, and `&&`.